

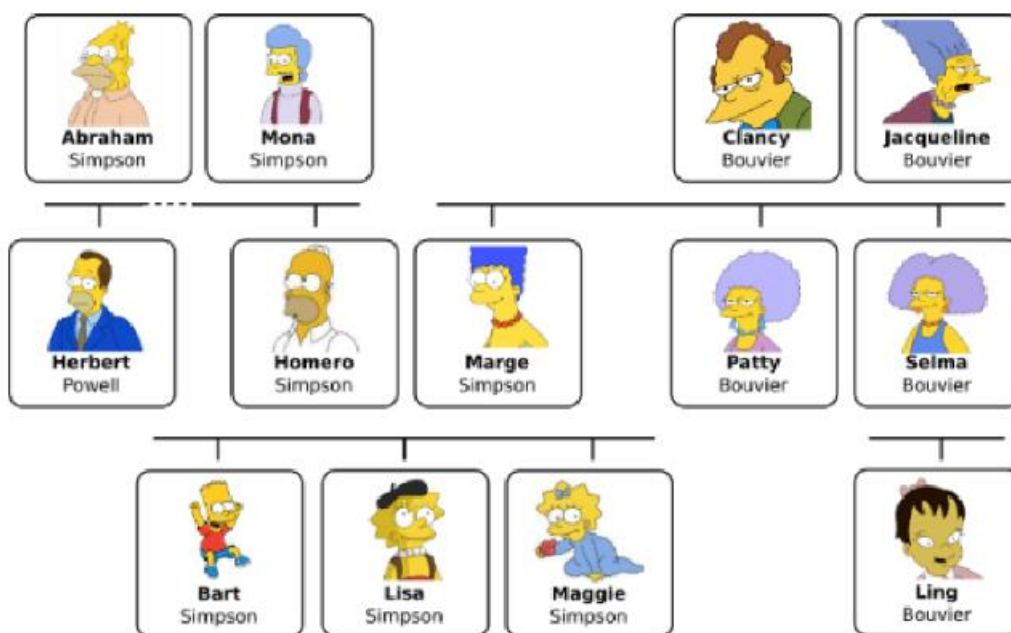
Entregable Actividad 06: Casos con estructuras y listas

Numeral 1 de la actividad 1 pero utilizando estructuras y listas.

Presentado por:

- Ramiro Pinchao
- Kenneth Rodríguez
- Brayan Suárez

1. De acuerdo a la siguiente imagen de árbol genealógico, construya una lógica de predicados donde las relaciones directas se generen por hechos y las relaciones de más de una generación se obtengan mediante reglas. Ejemplo: padre(homero,bart) el anterior es un hecho y corresponde a una relación directa, mientras que abuelo(X,bart) puede ser una consulta hecha al programa, donde la relación abuelo (que es de más de 1 generación) debe ser obtenida por reglas, no por hechos.



Solución:

Código empleando estructuras y listas:

```
progenitor_de( padre( abrahamSimpson ), hijo( [ herbertPowel, homeroSimpson ] )
).
progenitor_de( padre( abrahamSimpson ), hija( [ ] ) ).
progenitor_de( padre( clancyBouvier ), hija( [ margeSimpson, pattyBouvier,
selmaBouvier ] ) ).
progenitor_de( padre( clancyBouvier ), hijo( [ ] ) ).
progenitor_de( padre( homeroSimpson ), hijo( [ bartSimpson ] ) ).
progenitor_de( padre( homeroSimpson ), hija( [ lisaSimpson, maggieSimpson ] ) ).
progenitor_de( madre( monaSimpson ), hijo( [ herbertPowel, homeroSimpson ] ) ).
progenitor_de( madre( monaSimpson ), hija( [ ] ) ).
progenitor_de( madre( jacquelineBouvier ), hija( [ margeSimpson, pattyBouvier,
selmaBouvier ] ) ).
progenitor_de( madre( jacquelineBouvier ), hijo( [ ] ) ).
progenitor_de( madre( margeSimpson ), hijo( [ bartSimpson ] ) ).
progenitor_de( madre( margeSimpson ), hija( [ lisaSimpson, maggieSimpson ] ) ).
progenitor_de( madre( selmaBouvier ), hija( [ lingBouvier ] ) ).
progenitor_de( madre( selmaBouvier ), hijo( [ ] ) ).
```

```
abuelo_de(X, Y):-
    progenitor_de( padre(X), hijo(Hijos) ), member( Z, Hijos ),
    (
        (
            ( progenitor_de( padre(Z), hijo( Hijos2 ) ), member( Y, Hijos2 ) ) ; (
progenitor_de( padre(Z), hija( Hijos2 ) ), member( Y, Hijos2 ) )
            )
        )
    ;
    (
        ( progenitor_de( madre(Z), hijo( Hijos2 ) ), member( Y, Hijos2 ) ) ; (
progenitor_de( madre(Z), hija( Hijos2 ) ), member( Y, Hijos2 ) )
        )
    ).
```

```
abuela_de(X, Y):-
    progenitor_de( madre(X), hijo(Hijos) ), member( Z, Hijos ),
    (
        (
            ( progenitor_de( padre(Z), hijo( Hijos2 ) ), member( Y, Hijos2 ) ) ; (
progenitor_de( padre(Z), hija( Hijos2 ) ), member( Y, Hijos2 ) )
            )
        )
    ).
```

```

    )
;
    (
        ( progenitor_de( madre(Z), hijo( Hijos2 ) ), member( Y, Hijos2 ) ) ; (
progenitor_de( madre(Z), hija( Hijos2 ) ), member( Y, Hijos2 ) )
        )
    ).

```

hermano_de(X, Y):-
 progenitor_de(P, hijo(Hijos)), progenitor_de(P, hija(Hijas)), member(X, Hijos
),
 (member(Y, Hijos) ; member(Y, Hijas)), X \= Y.

hermana_de(X, Y):-
 progenitor_de(P, hijo(Hijos)), progenitor_de(P, hija(Hijas)), member(X, Hijas
),
 (member(Y, Hijos) ; member(Y, Hijas)), X \= Y.

tio_de(X, Y):-
 (hermano_de(X, Padre), progenitor_de(padre(Padre), hijo(Hijos)),
 progenitor_de(padre(Padre), hija(Hijas)), (member(Y, Hijos) ; member(Y, Hijas)
));
 (hermana_de(X, Padre), progenitor_de(padre(Padre), hijo(Hijos)),
 progenitor_de(padre(Padre), hija(Hijas)), (member(Y, Hijos) ; member(Y, Hijas)
));
 (hermano_de(X, Madre), progenitor_de(madre(Madre), hijo(Hijos)),
 progenitor_de(madre(Madre), hija(Hijas)), (member(Y, Hijos) ; member(Y, Hijas
)));
 (hermana_de(X, Madre), progenitor_de(madre(Madre), hijo(Hijos)),
 progenitor_de(madre(Madre), hija(Hijas)), (member(Y, Hijos) ; member(Y, Hijas
))).

primo_de(X, Y):-
 (tio_de(Padre, Y), progenitor_de(padre(Padre), hijo(Hijos)),
 progenitor_de(padre(Padre), hija(Hijas)), (member(X, Hijos) ; member(X, Hijas)
));
 (tio_de(Madre, Y), progenitor_de(madre(Madre), hijo(Hijos)),
 progenitor_de(madre(Madre), hija(Hijas)), (member(X, Hijos) ; member(X, Hijas
))).

